Day3 笔记：路径、权限、目录整理

日期：2026-10-13　工具：Git Bash（VS Code 内置终端）

一、今日目标

- ☐ 搞懂绝对路径 vs 相对路径
- ☐ 认识 `.` `..` `~` 三个特殊符号
- ☐ 看懂文件权限（`ls -l` 开头那串字母）
- ☐ 了解 `chmod`，并知道它在 Windows 上的局限
- ☐ 把 day2 的三个练习目录整理进 `D:\Learn_code\day2\`

二、绝对路径 vs 相对路径

核心区别：看开头有没有 `/`

类型	特征	例子	从哪算起
绝对路径	以 `/` 开头	`/d/Learn_code/day2_practice`	从根目录 `/` 开始
相对路径	不以 `/` 开头	`day2_practice`	从**当前所在位置**开始

一句话记住

> **绝对路径 = 完整地址，从哪说都不会错。 相对路径 = "往前走两个路口"，换个地方说就错了。**

今天的真实对比（我亲手踩的坑）

```
# 失败：开头的 / 漏了
$ cd d/Learn_code/day2_practice
bash: cd: d/Learn_code/day2_practice: No such file or directory

# 成功：加上开头的 /
$ cd /d/Learn_code/day2_practice
（无报错，提示符变了）
```

失败原因：不写 `/` 开头，bash 以为是"从当前位置往下找 `d/Learn_code/day2_practice`"。

而我当时在 `/d/Learn_code` 里，那里并没有一个叫 `d` 的文件夹。

什么时候用哪个

场景	建议用
写脚本	**绝对路径**（脚本在哪儿跑都不会错）
手动操作，懒得打长路径	相对路径（反正我知道自己在哪）
不确定自己在哪	先 `pwd`，再用相对路径

三、三个特殊符号：`.` `..` `~`

`.` —— 当前这个文件夹

```
$ ls .
day2_exercise/   day2_homework/   day2_practice/   ...
# 和光打 ls 效果一模一样
```

```
$ cd .
# 什么也不发生：你走到的就是你现在站的地方
```

用途：主要在路径中间表示"放到这儿"。比如以后常用的：

```
cp ../hello.py .        # 把上一层的 hello.py 复制到当前文件夹
```

`..` —— 上一层文件夹

```
$ pwd
/d/Learn_code/day2_practice

$ cd ..
$ pwd
/d/Learn_code              # 回上一层了
```

```
$ ls ..
7zip/  360Safe/  BaiduNetdiskDownload/  ...    # 列出上一层的内容
```

可以叠加：

```
cd ../..        # 往上两层
```

`~` —— 家目录（你的用户文件夹）

```
$ cd ~
$ pwd
/c/Users/马钧文
```

注意提示符的偷懒行为：

- 你在家目录时，提示符只显示 `~`（不写全地址）
- 但 `pwd` 会老实报出 `/c/Users/马钧文`
- **`~` 只是外号，两者是同一个地方**

`$HOME` —— 装着家目录地址的变量

```
$ echo $HOME
/c/Users/马钧文
```

`echo` = 把这个东西原样打出来。`$HOME` 是个变量（就是"盒子"），里面装着家目录的真实地址。

`~` 之所以能用，就是因为系统知道 `$HOME` 在哪。

它们都是真实存在的文件夹

```
$ ls -a
./  ../
```

加上 `-a`（all）之后，`.` 和 `..` 会露出来——**证明它们是真实文件夹，不是键盘符号。**

系统在每个文件夹里都偷偷放了这两个。

四、Windows 路径在 Git Bash 里怎么写

Windows	Git Bash
`C:\`	`/c/`
`D:\`	`/d/`
`C:\Users\马钧文`	`/c/Users/马钧文`
`D:\Learn_code`	`/d/Learn_code`

规律：盘符变小写，反斜杠 `\` 全换成正斜杠 `/`。

五、`ls` 常用选项

`ls` = list（列出）

命令	全称	作用
`ls`	—	列出当前目录
`ls -l`	**l**ong	长格式：权限、属主、大小、时间
`ls -a`	**a**ll	连隐藏文件（`.` 开头）一起列
`ls -h`	**h**uman	大小显示成 K/M/G，不显示字节数
`ls -r`	**r**everse	倒序排列
`ls -t`	**t**ime	按修改时间排序

可以叠写：`ls -lh`、`ls -la`、`ls -lha`

查看所有选项：

```
ls --help
```

> 单横杠接单字母（`-l`），双横杠接单词（`--help`）——这是 Linux 的老规矩。

输出颜色的含义

- **蓝色** = 文件夹（末尾带 `/`）
- **黑色/白色** = 文件

六、读懂 `ls -l` 的输出

```
drwxr-xr-x  1  马钧文  197610  0  10月 13 14:50  day2_exercise/
-rw-r--r--  1  马钧文  197610 18  10月 14 10:45  demo.sh
|└──┬──┘  |  |      |      |  |       |              |
|   |     |  |      |      |  |       |              └ 名字
|   |     |  |      |      |  |       └ 修改时间
|   |     |  |      |      |  └ 大小（字节）
|   |     |  |      |      └ 属组 ID
|   |     |  |      └ 属主（谁的文件）
|   |     |  └ 链接数（先不用管）
|   |     └ 权限（9 个字母，见下）
└ 类型：d = 文件夹，- = 文件
```

那 9 个字母怎么读

```
r w x   r - x   r - x
└─┬─┘   └─┬─┘   └─┬─┘
  你      同组人   其他人
```

字母	含义
`r`	read，可读
`w`	write，可写
`x`	execute，可执行
`-`	没有这个权限

你的实际例子：

- 文件夹：`drwxr-xr-x` → 是文件夹，你有读写执行，别人只读+执行
- 文件：`-rw-r--r--` → 是文件，你能读写，别人只能读

观察到的现象：文件夹比文件**多一个** `x`。因为"进入一个文件夹"这个动作本身就需要 `x` 权限。

七、`chmod` 与它的 Windows 局限

命令本身

`chmod` = **ch**ange **mod**e（改变权限模式）

写法	意思
`chmod +x 文件`	给文件**加上**可执行权限
`chmod -x 文件`	**去掉**可执行权限
`chmod -R 755 文件`	递归设置成 755

`+` 是加，`-` 是减，`-R` 是 recursive（递归，对文件夹内所有东西生效）。

⚠️ 今天的重点发现：在 Windows 上它不生效

我亲手验证的实验：

```
$ ls -l demo.sh
-rw-r--r-- 1 马钧文 ... demo.sh          # 没有 x

$ chmod +x demo.sh                         # 执行，无报错

$ ls -l demo.sh
-rw-r--r-- 1 马钧文 ... demo.sh          # 还是没 x！一点没变

$ chmod -R 755 demo.sh                     # 换一种写法再试

$ ls -l demo.sh
-rw-r--r-- 1 马钧文 ... demo.sh          # 仍然没变
```

结论：

> **在 Windows（NTFS 文件系统）上，Git Bash 的 `chmod` 改不动权限位。**

为什么

- `rwx` 这套权限是 **Linux/Unix 的规矩**
- **Windows 用的是完全另一套**（NTFS 权限）
- Git Bash 为了让 Linux 命令能用，**假装显示** `-rw-r--r--`，但改的时候**没有地方可改**

还有一个连带发现

```
$ ./demo.sh
我是脚本                # ✅ 居然跑成功了！
```

在真正的 Linux 上，文件没有 `x` 权限时 `./demo.sh` 会报 `Permission denied`。

但 Windows 上**不检查这个**，所以照样能跑。

那还学它干什么

环境	chmod 有用吗
Windows + Git Bash	❌ 没用（但敲了也不报错，无害）
真实 Linux 服务器	✅ **非常有用**

以后一定会遇到：把 `.sh` 脚本传到 Linux 服务器，执行时报 `Permission denied`——那时候 `chmod +x` 就是解药。

记住这条更大的道理

> **Windows 上练语法，Linux 上才是真规则。** 在这台机器上看到的行为，不一定是"编程的真相"。

八、`mv`：移动 / 改名

用法	效果
`mv 文件 目标文件夹/`	**移动**（从 A 到 B）
`mv 旧名 新名`	**改名**

今天做的事

```
mkdir -p day2
mv day2_practice day2/
mv day2_homework day2/
mv day2_exercise day2/
```

一个反常的点：`mv` 移动文件夹不需要 `-r`

你 Day2 学过 `cp` 复制文件夹要 `-r`。但 `mv` 不要。

> **记忆**：搬家不用打包（`mv` 不要 `-r`），复印要一张张来（`cp` 要 `-r`）。

`mv` vs `rm`（重要区别）

命令	效果	能恢复吗
`mv`	搬家，东西还在	✅ 移错了还能移回来
`rm`	扔掉	❌ **真删除，回收站里没有**

所以 `mv` 不可怕，`rm` 要小心。

关于末尾那个 `/`

```
mv a day2        # day2 已存在 → 移进去（和加 / 效果一样）
mv a day2        # day2 不存在 → 把 a 改名成 day2
mv a day2/       # day2 不存在 → 报错，提醒你搞错了
```

（上面第 1、2 行写法相同但结果不同——**因为结果取决于 `day2` 在硬盘上存不存在，不取决于你怎么写**。）

加 `/` 的作用：明确告诉 bash "这是文件夹"，还能提前暴露错误。

但 `/` 不是必须的——bash 判断是文件还是文件夹，**是去看硬盘上那个东西实际是什么**，不是看斜杠。

> **`/` 是提示，不是语法要求。**

九、今天卡住的地方 & 怎么解决的

> ✍️ **这一段留给你自己写**（写大白话就行，不用讲究格式）

卡点 1：

怎么解决的：

卡点 2：

怎么解决的：

十、今天最值钱的三个收获

> ✍️ **用你自己的话各写一句**

1. 关于路径：

2. 关于报错：

3. 关于"先看一眼再动手"：

十一、命令速查表

```
# 路径
pwd                         # 我在哪（绝对路径）
cd /d/Learn_code            # 绝对路径跳转
cd day2                     # 相对路径跳转
cd ..                       # 上一层
cd ~                        # 回家目录
cd .                        # 原地不动（没什么用）
echo $HOME                  # 家目录的真实地址

# 查看
ls                          # 列出当前目录
ls -l                       # 详细信息（权限）
ls -a                       # 连隐藏文件
ls -lh                      # 详细 + 人类可读大小
ls day2                     # 列出指定目录
ls ..                       # 列出上一层

# 权限
ls -l 文件                  # 看权限
chmod +x 文件               # 加可执行权限（Windows 上无效）
chmod -R 755 文件           # 递归设置

# 目录操作
mkdir -p day2               # 建目录（-p 可建多层）
mv 源 目标/                 # 移动
mv 旧名 新名                # 改名
rm -r 目录                  # 删除目录（危险！）
```

十二、踩坑记录（更新中）

坑	正确做法
`cd d/Learn_code/...` 报错	绝对路径**要**以 `/` 开头
打错字导致 `No such file or directory`	看清报错里冒号后面那串路径，对着真实名字比
`chmod` 没效果	Windows 上就是不生效，不是你的问题
`mv` 移动文件夹加了 `-r`	不需要，`mv` 从来不加 `-r`
不确定自己在哪	**先 `pwd` / `ls`，再动手**
不确定某步做了没有	`ls` **看一眼**，别靠记忆
终端用 ↑ 上箭头	调出上一条命令，改错字不用重敲

十三、Day3 完成清单

- ☐ 走通绝对路径与相对路径
- ☐ 认识 `.` `..` `~` `$HOME`
- ☐ 看懂 `ls -l` 的权限串
- ☐ 亲手验证 `chmod` 在 Windows 上的局限
- ☐ 三个 day2 目录整理进 `day2/`
- ☐ 删除实验目录 `day3_test`
- ☐ 验收命令全部通过
- ☐ 写笔记（正在做）
- ☐ Git 提交

十四、下一天预告（Day4）

第一个 Python 程序：打印、变量、输入输出、注释。

写一个"打招呼 + 计算年龄"的小程序。